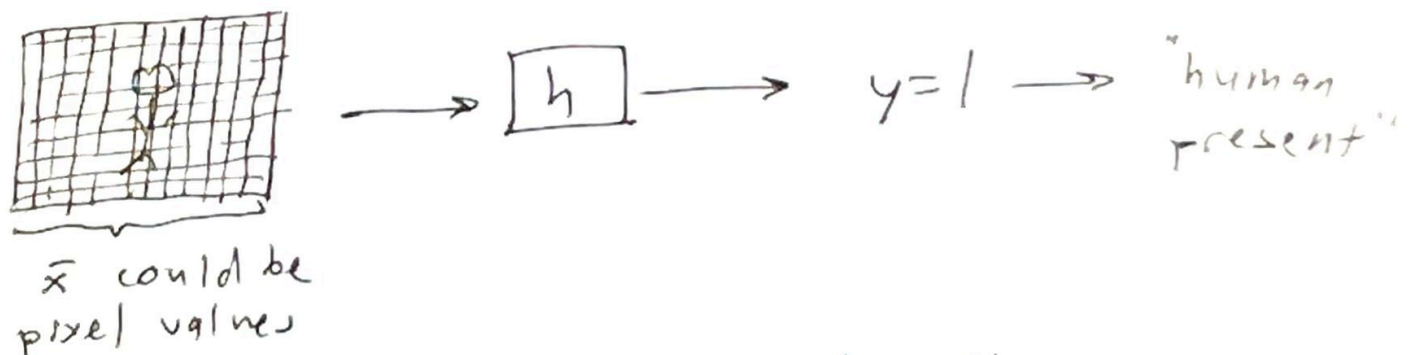


Classification

Let y denote a class. y takes values from a finite set $\{1, 2, \dots, M\}$.

Let $h(\bar{x})$ denote a model. In regression problems $h(\bar{x})$ would ~~also~~ output a real number; however, in classification the output is an integer denoting a class.

$$h(\bar{x}) : \mathbb{R}^d \rightarrow \underbrace{\{1, \dots, M\}}_{\text{classes}}$$



$\mathbb{R}^d$ may be ~~imag~~ integers as would be the case for an image represented as discrete pixels.

Figure 1

Often we break the problem into more than one.

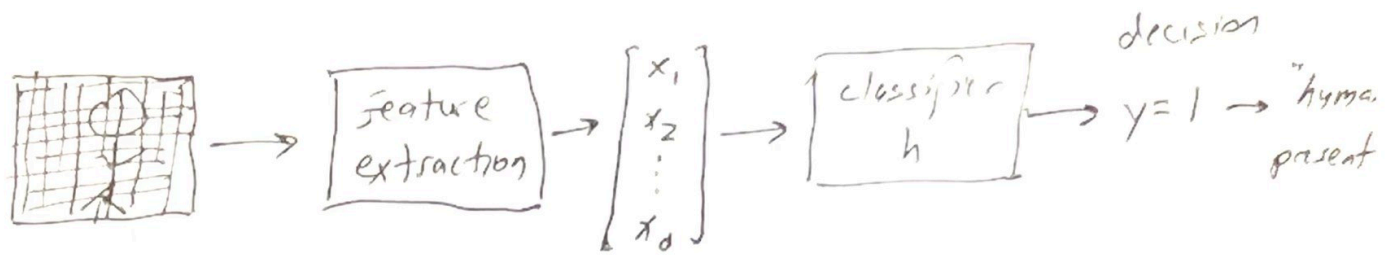
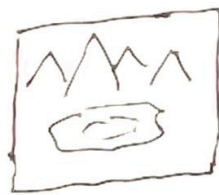


Figure 2. Feature Extraction & classification

Error

h errs on $\bar{x}$ if $h(\bar{x}) \neq y$ ↖ ground truth



$\bar{x}$
= image of
scenery with
no human present

$\longrightarrow h(x) \longrightarrow 1 \neq y$

Declaration
that a human
is present when
there is none.

$\nwarrow$ h errs on $\bar{x}$.

Good vs. Bad Classifiers

The best possible classifier never errs.

In some problems we may be more tolerant of errors in one direction (e.g., declaring people are present when they are not) which would be an example of biasing the output in favor of recall.

In some cases, we may prefer that the classifier only decide on a particular class when the classifier is fairly certain even if this means sometimes failing to recognize the class. For example, our "human present" classifier would decide "no human present" unless it is fairly certain there is a human present. This represents bias in favor of "precision."

A classifier that minimizes error is a classifier that maximizes "accuracy."

accuracy = fraction of classifications that are correct

$$= \frac{TP + TN}{\text{all classifications}}$$

error = fraction of incorrect classifications

$$= \frac{FP + FN}{\text{all classifications}}$$

$$\text{error} = 1 - \text{accuracy}$$

Assume all observations are generated by a joint distribution.

$$(\bar{x}, y) \sim P(\bar{x}, y)$$

feature vector $\nearrow$ $\bar{x}$ scalar $\nearrow$ y means "obeys the distribution defined by"
} joint probability density function.

From Bayes' Law

$$P(\bar{x}, y) = P(\bar{x} | y) P(y)$$

$$\text{let } y \in \{\omega_1, \omega_2, \dots, \omega_M\}$$

$\nearrow$ class label ω_1 $\nwarrow$ M is the number of classes.

$P(y = \omega_i)$ = probability of an occurrence of class ω_i given no conditions. The probability ω_i would be encountered under "typical" conditions.

= "prior" probability of ω_i .

$P(\bar{x} | \omega_i)$ = likelihood of an observation $\bar{x}$ when it is an instance of class ω_i .

Training Data sets

Ex:

 $w_1 =$ "person appears in the image" $w_2 =$ "no person appears in the image"

If the training data set is an unbiased representation of the data that will be input to the classifier then we can estimate the "prior probability" for each class i in $\{1, \dots, M\}$ as

$$P(w_i) \approx \frac{\text{number of instances in training data labelled } w_i}{\text{size of the training data set}}$$

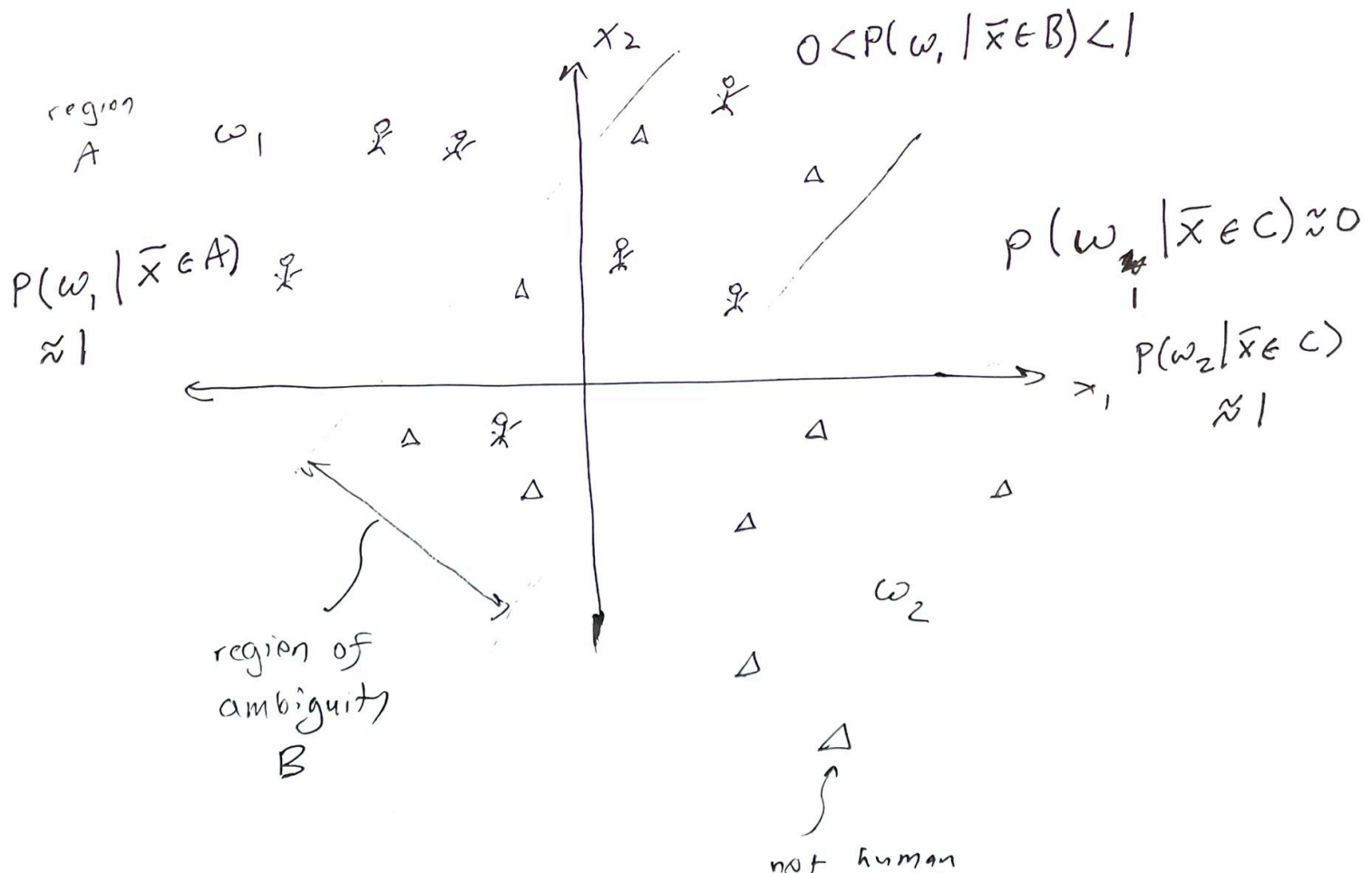


Figure 3: Region of Ambiguity

$p(y = \omega_i)$ = probability of an occurrence of
class ω_i

= "prior" probability of ω_i

$p(\bar{x} | \omega_i)$ = likelihood of an observation $\bar{x}$
when $\bar{x}$ came from an instance of
class ω_i .

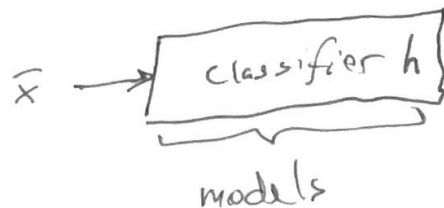
If the training data set is an unbiased representation
of the samples the classifier will encounter in regular
usage then, the prior probability can be directly
estimated from the training data set.

$$p(y = \omega_i) = p(\omega_i) \approx$$

shortened way
of saying the
same thing as
 $p(y = \omega_i)$

number of ~~instances~~ ^{samples} of class ω_i
within the training data set

number of samples in the
entire training data set



$$p(\bar{x} | \omega_i)$$

Figure 2

A good machine learning algorithm learns the shape
of the distribution of the features for each class.
More sophisticated models can learn the shape of
more complex distributions.

For any classifier $h(\bar{x})$, we can define its error rate

$$p(\text{error}) = \int_{\bar{x} \in \mathbb{R}^n} p(\text{error}, \bar{x}) d\bar{x}$$

Due to Bayes' Law, we can rewrite this as

$$p(\text{error}) = \int_{\mathbb{R}^n} p(\text{error} | \bar{x}) p(\bar{x}) d\bar{x}$$

Binary Classification

with binary classification, there are two classes

$$y \in \{\omega_1, \omega_2\}$$

$p(\omega_1 | \bar{x})$ = probability a sample with feature vector $\bar{x}$ is a member of class ω_1 .

$p(\omega_2 | \bar{x})$ = probability a sample with feature vector $\bar{x}$ is a member of class ω_2 .

It is possible that the exact same feature vector can have non-zero probabilities of being in more than one class. This occurs when the features are not distinguishing enough to fully differentiate between ω_1 & ω_2 .

Where we draw the boundary between where $h(\bar{x})$ predicts w_1 vs. w_2 is called the decision boundary.

$\forall \bar{x} \in A$: $h(\bar{x})$ could trivially predict w_1 .

$\forall \bar{x} \in C$: $h(\bar{x})$ could trivially predict w_2 .

$\forall \bar{x} \in B$: it is more complicated.

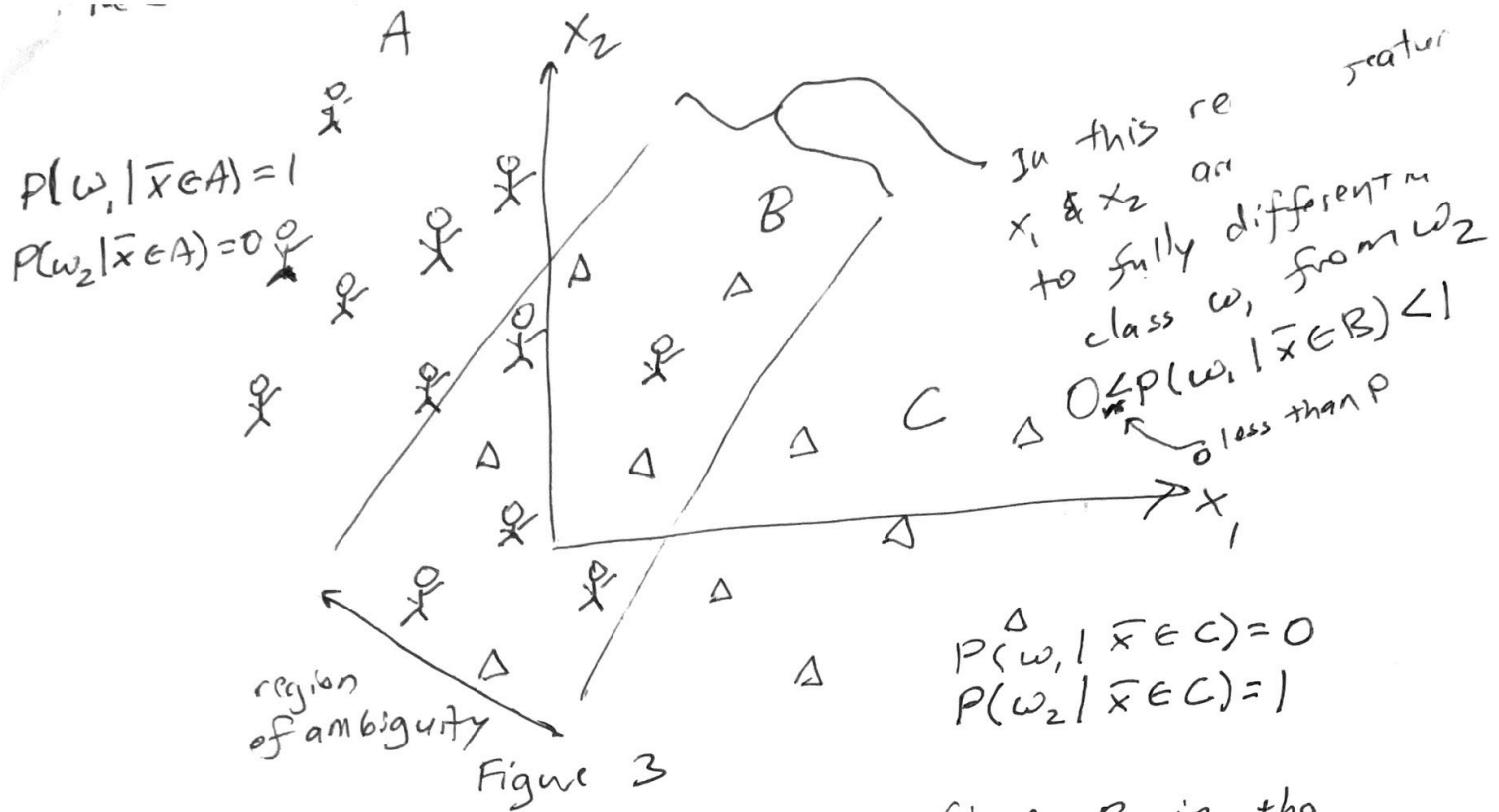
For region B, no classifier can always be right. If we want a classifier that minimizes error, the optimal classifier is one that perfectly models $p(w_1 | \bar{x})$ and $p(w_2 | \bar{x})$, and the model always picks the class with the least probability of being wrong.

The optimal classifier (in this case the maximally accurate classifier) always chooses the class with the minimum probability of error.

$$P(\text{error} | \bar{x}) = \min \{ p(w_1 | \bar{x}), p(w_2 | \bar{x}) \}$$

$$\text{Let } p^*(\text{error}) = \min_{h(x)} p(\text{error})$$

↑
optimal error rate.



we used ~~essentially~~ the same figure 3 in the previous lecture, but we then had midterm review and a midterm, so let's consider it again. When $\bar{x}$ falls in the region of ambiguity (denoted as region B in Figure 3), the feature vector does not fully differentiate between w_1 & w_2 .

If $h(\bar{x})$ predicts $\bar{x}$ as w_2

$$P(\text{error} | \bar{x}) = 1 - P(w_2 | \bar{x}) = P(w_1 | \bar{x})$$

if $h(\bar{x})$ predicts $\bar{x}$ as w_1

$$P(\text{error} | \bar{x}) = 1 - P(w_1 | \bar{x}) = P(w_2 | \bar{x})$$

Combine the above

$$P(\text{error} | \bar{x}) = \begin{cases} P(w_1 | \bar{x}) \\ P(w_2 | \bar{x}) \end{cases}$$

if $h(x)$ should predict w_2
if $h(x)$ should predict w_1

Is it possible to improve the optimal?

Yes, but only by changing features. We use better features, ~~that~~ i.e., features that better differentiate between w_1 and w_2 .

Even if we perfectly model $p(w_1|\bar{x})$ and $p(w_2|\bar{x})$

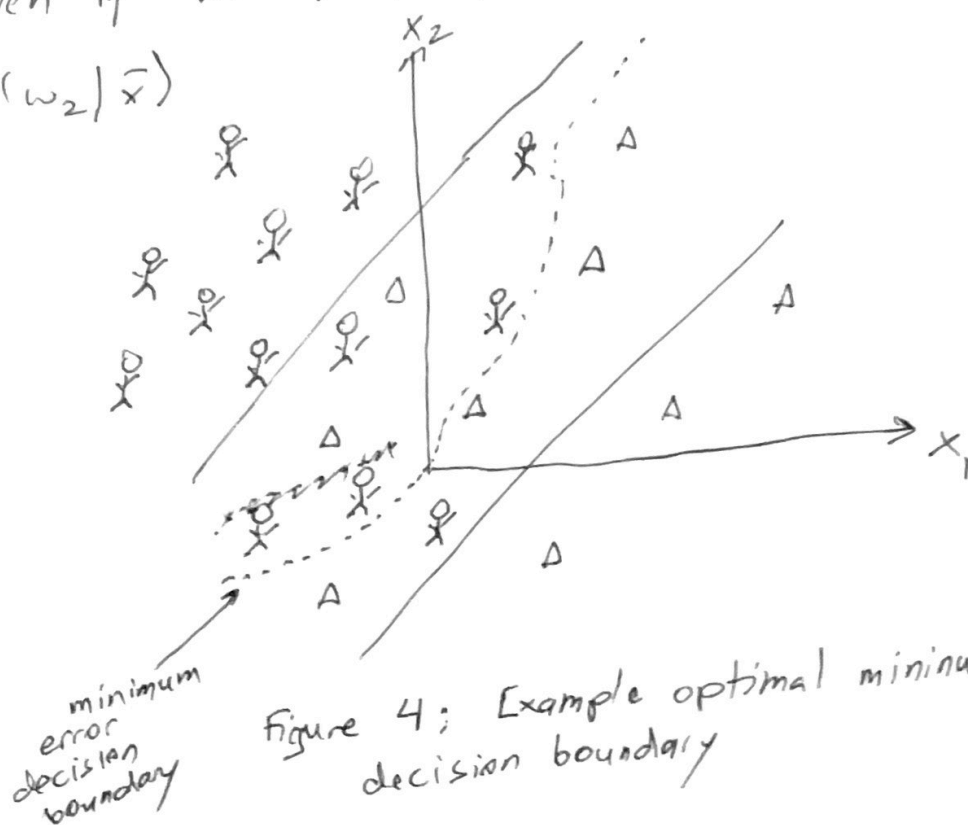


Figure 4: Example optimal minimum error decision boundary

In Figure 4, assuming our training data is perfectly representative of the distributions $p(w_1|\bar{x})$ and $p(w_2|\bar{x})$, the minimal error decision boundary could be highly non-linear.

For any decision boundary (i.e., for any $h(x)$)

$$p(\text{error}) = \int_{\mathbb{R}^d} p(\text{error}|\bar{x}) d\bar{x} \geq \int_{\mathbb{R}^d} \min(p(w_1|\bar{x}), p(w_2|\bar{x})) p(\bar{x}) d\bar{x}$$

The optimal classifier (in the sense of minimizing error) is usually unachievable, especially when the minimum error decision boundary is complicated.

Just ~~because~~ because the decision boundary is complicated doesn't mean we can't obtain good performance. We can use more complex models based on neural networks.

For all $\bar{x}$ that is not near the decision boundary we can sometimes improve computational cost by using simple approximations.

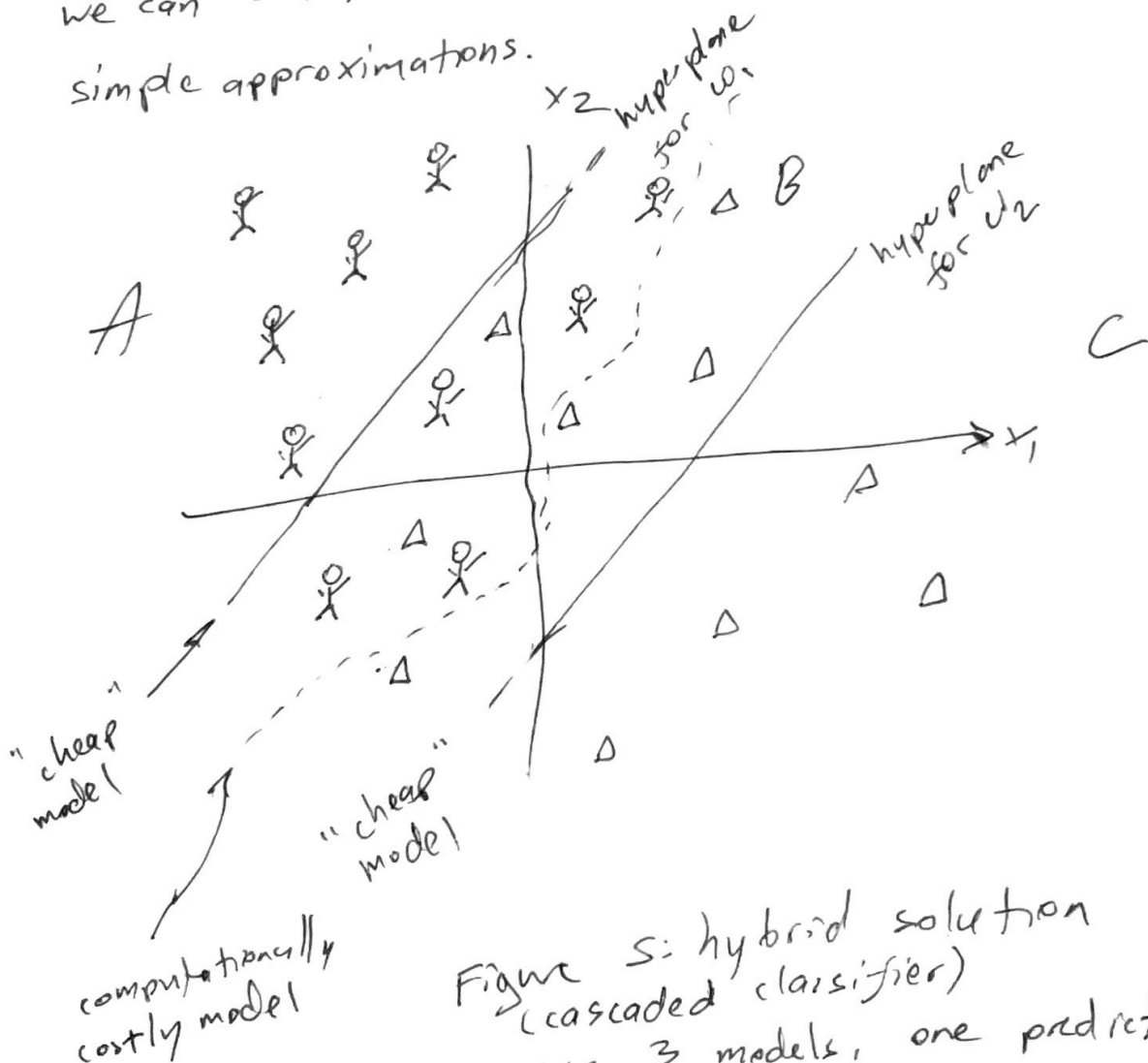


Figure 5: hybrid solution (cascaded classifier)

In Figure 5, we use 3 models, one predicts w_1 (region A) for class w_2 (i.e., region C) and third costlier algorithm is used within the region of ambiguity.